Polymorphic Type Inference

Parametric Polymorphism

Types that are "for all." Example: `map` in SML says "for any types A and B, give me an A→B function and a list of A's, I'll return a list of B's." No info about the types needed to type-check.

How languages handle "for all" being too constraining:

- **C++**: Only type-checks on types you actually use ("not truly for all T, for all T's you use"). Templates are specialized per type.
- **Java**: Type bounds – `for all T extends Comparable`. Type-check body once, instantiate only with subtypes of bound.
- **SML**: `'a` = any type. `''a` = equality types (where `=` is defined). For generic structures needing comparison (ordered maps), use **functors**: give a signature defining operations, it creates a structure.

Type Inference Algorithm

1. Assign fresh type variables to every parameter and return type.
2. Recurse down AST as in normal type checking.
3. On the way back up, instead of checking "are these equal?", **unify** the types you have with the types you expect.
4. Unification may fail (ill-typed), do nothing (already match), or produce mappings (new info about type vars).

Unification Algorithm

Base cases:

- Type var with type → return that mapping (e.g., `'a` with `int` gives {`'a`→int})
- Same type with itself → return ∅ (nothing to do)
- Constructor mismatch (arrow vs tuple, int vs string) → FAIL ("Tycon mismatch")
- Unary constructors like `list`: strip off `list` and unify argument

Recursive case:

```
unify(T1 ?? T2, S1 ?? S2) =
  let m1 = unify(T1, S1)
      T2' = apply(m1, T2) (* MUST apply *)
      S2' = apply(m1, S2) (* before continuing *)
      m2 = unify(T2', S2')
  in  combine(m1, m2)
```

CRITICAL: Before unifying the right parts, you MUST apply the substitution from unifying the left parts. Otherwise you get inconsistent results.

Concrete examples:

- `unify(int*'a, int*string)` → {'a→string}
- `unify(int->'a, 'b*string)` → FAIL (arrow vs tuple, tycon mismatch)
- `unify('a*'a, int*string)`: left gives 'a→int; apply: unify(int,string) → FAIL
- `unify('a list, int list)` → strip list, unify('a, int) → {'a→int}

Worked Example: Type Inference for len

`fun len list = case list of [] => 0 | y::z => 1 + len z`
Step 1 – Assign type vars: list:'a, return:'b, y:'c, z:'d. len : 'a→'b.
Step 2 – Unify from bottom up:
`len(z)`: z has type 'd, len expects 'a. Unify 'd with 'a → 'a='d.
`1+len z`: plus needs int*int→int. len z returns 'b. Unify 'b with int → 'b=int.
Pattern `y::z`: cons has type 'e*'e list→'e list. Unify 'e*'e list with 'c*'d → 'e='c, 'd='c list.
Nil `[]`: type 'f list. Unify with 'c list → 'f='c.
Both cases match 'c list and produce int.
list had type 'a = 'd = 'c list. Return type 'b = int.
Result: len : 'c list → int (i.e., 'a list → int).

Practice Final Q1 – Type Inference

`fun f(w,x) = case x of [] => [] | y::z => w(y)::f(w,z)`
Assign: w:'a, x:'b, return:'c, y:'d, z:'e.
Unification table:
- x with [] → 'b = 'f list
- (y,z) with cons arg → 'd='h, 'e='h list
- x with cons result → 'f='h
- w as fn taking y → 'a = 'h→'i
- (w(y), f(w,z)) with cons → 'i='j, 'c='j list
- 1st case = 2nd case → 'k list = 'j list → 'k='j
- f's arg types consistent: ('h→'j)*'h list = ('h→'j)*'h list ✓

Final type: $f : (\text{'h} \to \text{'j}) \times \text{'h list} \to \text{'j list}$
This is `map`! Takes a function and a list, applies function to each element.
Part 2: `fun f(x) = f` fails the **occurs check** (see below).

Occurs Check

When mapping `'b` to type T, check that `'b` does NOT appear inside T. Otherwise the type would be infinite.

Example: `fun f x = f`. f:'a→'b. Body is f itself ('a→'b). Must unify 'b with 'a→'b. But 'b appears inside 'a→'b → infinite type → **FAIL**.

Hungry Types (Workaround for Occurs Check)

```
datatype 'a hungry = F of ('a -> 'a hungry)
```

This is a valid, finite SML datatype. Now `fun f x = F(f)` works: f:'a→'a hungry. F expects 'a→'a hungry, returns 'a hungry. Everything checks out.

Why "hungry": Pattern match F to get f, apply to a value, get another 'a hungry, keep feeding forever. Used for: print sinks, random generators, Y combinator.

SML datatypes allow recursive types via constructors. The occurs check only fails for bare type variables, not datatype-wrapped recursion.

Options Gotcha

`case ... of ... => NONE | ... => 42` FAILS: NONE has type 'a option, 42 has type int. Can't unify option with int (tycon mismatch).

Fix: `SOME 42` so both branches are int option.

Subtyping

Covariance: if you can **read** it ⇒ same direction.

Contravariance: if you can **write** it ⇒ opposite direction.

Invariant: read+write ⇒ no subtyping allowed.

Functions: covariant in return type (you read the return), contravariant in argument type (you write the argument into the function).

If Lion <: Cat:

- Lion list <: Cat list (covariant – lists are read-only in SML)
- Lion ref ≮: Cat ref (**invariant** – refs are read+write. Could write a Frog into Cat ref, then read expecting Lion → type error)
- Cat→int <: Lion→int (contravariant in arg: function expecting Cat can handle anything given Lion, but a Lion→int function can't handle arbitrary Cats)
- If it were `const ref` (read-only), then covariance would apply

SML: `'a`=any type, `''a`=equality types. Complex constraints → **functors** (give a signature, get a structure).

MIPS Registers & Calling Convention

Reg	Name	Purpose
r0	zero	Hardwired to 0
r1	AT	Assembler temp (pseudo-instrs like BLT decompose to SLT+branch using r1)
r2–3	v0–v1	Return values
r4–7	a0–a3	Arguments (¿4 on stack)
r8–15	t0–t7	**Caller-saved** temps
r16–23	s0–s7	**Callee-saved** (must restore)
r24–25	t8–t9	Caller-saved temps
r26–27	k0–k1	Reserved (OS/kernel)
r28	gp	Global pointer (globals = GP+offset)
r29	sp	Stack pointer
r30	fp	Frame pointer
r31	ra	Return address

Usable registers: ∼25 (excluding r0, r1, r26–28).

T vs S analogy: Like treadmills at the gym. T regs = you wipe down before use (caller saves). S regs = previous user wipes after (callee saves/restores).

Function Prolog (before body)

1. `SP -= frame_size` (allocate stack frame)
2. Save old FP to stack
3. `FP = SP + frame_size` (set up new FP)
4. Save any S registers this function will use (only ones we clobber)

5. Save RA – **only if non-leaf** (leaf = calls no other functions)

Function Epilog (at end)

1. Restore saved S registers
2. Restore RA (if saved)
3. `SP = FP` or `SP += frame_size` (reset SP)
4. Restore old FP from stack
5. `jr $ra` (return)

Caller's Responsibilities

Before call: Save any T regs you need preserved. Move args into a0–a3 (extras on stack). Execute `jal target`.

After call: Restore T regs. Result in v0.

Why Frame Pointer Matters

1. **Fixed reference**: FP stays fixed throughout function. SP changes with `alloca()`, VLAs, nested call args. Vars at FP−offset (stable) vs SP+offset (varies).
2. **Variable-size frames**: C allows `int arr[n]` where n is runtime. SP moves unpredictably. FP remains fixed.
3. **Debugging**: Debuggers use FP chain (linked list of frames) to walk the call stack.

Can omit FP when no dynamic stack alloc → saves instructions at entry/exit, frees a register.

Static Links & Escaping Variables

Why Static Links?

Tiger allows nested functions where inner functions access/modify outer variables. These vars can't be in registers (no guarantee the register is intact levels deep). They must live in stack frames.

The naive approach (following old FP links / dynamic chain) fails because recursion creates unpredictable numbers of frames. The old FP follows the **dynamic call chain**, not the lexical nesting.

How Static Links Work

Each frame stores a **static link** (SL) pointing to its **statically enclosing function**'s frame (the function it is textually nested inside in source code).

Implementation in Tiger: SL passed as implicit first argument (a0). Stored in frame like an escaping variable.

Access variable N levels out: follow N static links, then access at known offset. Always exactly N regardless of recursion depth.

SL ≠ old FP: FP chain follows dynamic calls; SL follows lexical nesting.

Static Link Rules

- **Calling child** (function nested inside you): pass your **FP** (you are its static parent)
- **Calling sibling** (same nesting level): pass your **SL** = MEM(FP)
- **Calling self recursively**: pass your **SL** = MEM(FP) (same parent)
- **Calling function N levels out**: follow N static links

Practice Final Q2 – Static Links

Nesting: Level 0: top-level let. Level 1: `odd`, `f`. Level 2: `g`, `h` (inside f).
SL is stored in frame slot pointed to by FP.

Call	SL argument
f→g	**FP** (child – f is g's parent)
g→h	**MEM(FP)** (sibling – pass f's FP = g's SL)
g→g	**MEM(FP)** (recursive – same parent f)
h→f	**MEM(MEM(FP))** (2 up: h→f→top)
g→odd	**MEM(MEM(FP))** (2 up: g→f→top)

Deep nesting: `foo` inside `h` inside `g` inside `f`. To access `a` from f's frame in foo: follow 3 static links (foo→h→g→f). Always exactly 3 regardless of recursion depth.

Escaping Variables

A variable "escapes" if it can't go in a register:

- **Tiger**: used in a nested function → must live in stack frame
- **C**: address taken (`&x`) → needs memory location

Escape analysis: Recurse over AST. Map each variable to its declaration depth. When referenced at a deeper depth, mark as escaping. Non-escaping vars → hopefully in registers (faster).

Intermediate Representation (IR)

Why IR?

1. **N+M instead of N*M**: N languages, M ISAs. Without IR: N*M compilers. With IR: N front-ends + M back-ends.
2. **Incremental step**: AST→assembly is a huge leap. IR is halfway.

Feature	AST	IR	Assembly
Structure	Tree	Tree	Linear
Functions	Nested	Flat list of fragments	Top-level
Variables	Named	Infinite temps	∼25 regs
Scope	Nested/tree	Flat	Flat
Types	int,string,...	32-bit words	byte,half,word
Control flow	if/while/for	2-target CJUMP	1-target branch

IR Datatypes

Statements (side effects, no value):

- **SEQ(s1,s2)**: do s1 then s2
- **LABEL(l)**: declares label location
- **JUMP(e, label list)**: unconditional jump. List usually [l]. **Empty list** for `jr $ra` (end of function). Multiple labels for jump tables.
- **CJUMP(relop,e1,e2,lt,lf)**: conditional jump with **two targets** (true and false labels). relop: $<, \leq, >, \geq, =, \neq$. No fall-through in IR.
- **MOVE(dst,src)**: dst must be TEMP or MEM. MOVE to MEM = store. MOVE to TEMP = register write.
- **EXP(e)**: evaluate expression for side effects, discard value.

Expressions (produce a value):

- **BINOP(op,e1,e2)**: arithmetic/logic
- **MEM(e)**: memory at address e. In MOVE dst = store; elsewhere = load.
- **TEMP(t)**: register from infinite pool
- **ESEQ(s,e)**: do statement s for effect, then evaluate e for value. How expressions have side effects.
- **NAME(l)**: reference to a label (address)
- **CONST(n)**: integer constant
- **CALL(f,args)**: function call. f is usually NAME(label).

Design flaw: MOVE dst is type `exp` but can only be TEMP or MEM. Should have been a separate `location` type. Causes non-exhaustive match warnings.

Fragments

A top-level assembly entity. Two types: **string literal** (label + string data) or **function** (body + frame info). Translation outputs a list of fragments.

Ex / Nx / Cx Wrappers

Three constructors for translate output:

- **Ex** (expression): wraps IR exp – things evaluated for value (e.g., a+b)
- **Nx** (no-result): wraps IR stm – things done for effect (e.g., a:=3)
- **Cx** (conditional): wraps a `label*label→stm` function – things used for control flow (e.g., x<4). Given true/false labels, produces CJUMP.

Return whichever is natural: `a+b`→Ex, `a:=3`→Nx, `x<4`→Cx.

Conversions Between Ex/Nx/Cx

unEx(Ex(e)): just return e.

unEx(Nx(s)): ESEQ(s, CONST 0) – do the statement, evaluate to 0.

unEx(Cx(f)): Must turn conditional into 0/1 value:

```
r = newTemp(); t1 = newLabel(); f1 = newLabel()
```

```
Ex(ESEQ(seq[
  MOVE(TEMP r, CONST 1),  (* assume true *)
  f(t1, fl),              (* CJUMP *)
  LABEL fl,               (* false: *)
  MOVE(TEMP r, CONST 0),  (* set 0 *)
  LABEL t1                (* true: 1 stays *)
], TEMP r))
```

unCx(Ex(e)): CJUMP(e ≠ 0, true_label, false_label). "Implicitly if expr ≠ 0."

unNx(Ex(e)): EXP(e) – evaluate, discard.

Helper: `fun seq [] = EXP(CONST 0) | seq [s] = s | seq (s::rest) = SEQ(s, seq rest)`

Practice Final Q3 – IR Translation

x →InReg t_1, y →InReg t_2, z →InFrame -4, a →InReg t_3.
y := 1+x: `MOVE(TEMP` t_2`, BINOP(PLUS, CONST 1, TEMP` t_1`))`
a[y] := f(z): `MOVE(MEM(BINOP(PLUS, TEMP` t_3`, BINOP(TIMES, TEMP` t_2`, CONST 4))), CALL(LABEL(f), [MEM(BINOP(PLUS, FP, CONST −4))]))`
Left side: array base t_3 + index t_2 * word size 4. Right side: call f with z from frame at FP−4.
Record r.z: MEM(BINOP(PLUS, TEMP r, CONST 8)) – z is 3rd field, offset 8 (0-indexed, 4 bytes each).

Translating Control Flow

If-Then-Else

3 params: condition (unCx), then-clause (unEx), else-clause (unEx).

```
c' = unCx(cond); t' = unEx(then); e' = unEx(else)
r = newTemp; lt/lf/le = newLabels
Ex(ESEQ(seq[
  c'(lt, lf),           (* CJUMP *)
  LABEL lt,             (* true case: *)
  MOVE(TEMP r, t'),     (* store then-value *)
  JUMP(NAME le, [le]),  (* jump to end *)
  LABEL lf,             (* false case: *)
  MOVE(TEMP r, e'),     (* store else-value *)
  LABEL le              (* end *)
], TEMP r))
```

7 statements in the sequence. For statement-only if: use unNx, moves store 0 (harmless).

Expression sequences with side effects: Tiger `(x:=x+1; x+3)` translates to ESEQ: do assignment, then evaluate x+3.

While Loop

Naive (2 branches/iter – bad):

```
L_start: if NOT cond, goto L_end
         body
         goto L_start
L_end:
```

For 1M iterations: 1,000,001 conditional + 1,000,000 unconditional = 2,000,001 branches.

Optimized (1 branch/iter – good):

```
  JUMP(NAME L_test, [L_test])
L_body: body
L_test: CJUMP(cond, L_body, L_done)
L_done:
```

1 initial jump + 1,000,001 conditional = 1,000,002. Saves ∼1M branches!

Break: `JUMP(NAME(done_label), [done_label])`. Track with `label option`: NONE if outside loop, SOME(label) if inside.

For Loop – Max-Int Edge Case

Tiger for loops have **inclusive bounds**. If high=maxint, naive `i++` overflows → infinite loop.

Correct compilation:

```
i := low; h := high
if i > h goto L_done  (* zero iterations *)
L_body: body
  if i < h goto L_incr (* STRICT less-than *)
  goto L_done          (* i==h, last iter *)
L_incr: i := i + 1
  goto L_body
L_done:
```

Key: check $i < h$ (strictly) BEFORE incrementing. If $i = h$, we've done the last iteration – don't increment (would overflow).

Arrays

Layout: [size | elem0 | elem1 | . . .]. Return ptr to elem0. Size at MEM(arr−4).

Create: malloc((size+1)*4), store size at offset 0, increment ptr by 4, fill values.

Index: MEM(base + idx*4). Eval idx into temp first (avoid dup side effects).

Bounds check: 0 ≤ idx < MEM(arr − 4). Two CJUMPs. Fail → jump to exit label.

IMPORTANT: If using index multiple times (bounds check + access), move it to a register first! An expression like (x:=x+1; x) would increment twice if evaluated twice.

Records

malloc, MOVE each field to MEM at offset. Access: MEM(base + offset). Offset known at compile time. No bounds check.

Switch/Case Compilation

- **If-else chain**: O(n). Fine for small switches.
- **Binary search**: O(log n). Sort cases, binary search.
- **Jump table**: O(1). Array of labels indexed by case value. Great for **dense** cases (0,1,2,. . . ,n). Terrible for sparse (0, 9000, 57000).
- **Hybrid**: Binary search to find dense range, then jump table within range.

Why JUMP has a label list: For jump tables, exp = MEM(i*4+tableBase), label list = all possible targets so later phases can build the CFG.

Pattern Matching Compilation

SML datatypes: each constructor gets an integer tag (NONE=0, SOME=1). Data stored as struct with `int tag` + union of payloads. Pattern match → check tag (if-else on ints), extract payload, recurse for nested patterns.

Critical Rule

Never reuse an IR sub-tree. Dup labels = asm error. Dup side effects = bugs. Evaluate once, store in TEMP.

Purifying & Linearizing

Removing ESEQ (Purifying Expressions)

Goal: bubble all side effects to top level. Want statements then pure expressions (no side effects).

Rule 1 – Squish nested ESEQs: ESEQ(s1, ESEQ(s2, e)) → ESEQ(SEQ(s1,s2), e). "Do s1 then s2, then evaluate e."

Rule 2 – Push ESEQs outward through BINOPs: BINOP(+, ESEQ(s,e1), e2):

- If s and e2 are **independent** (s doesn't change e2's value): → ESEQ(s, BINOP(+,e1,e2))
- If NOT independent (or unknown): → capture e1 first: ESEQ(SEQ(MOVE(t,e1), s), BINOP(+,TEMP t,e2))

Safe version always works but increases register pressure (t lives across s).

Alias Analysis (Determining Independence)

If no memory involved: Check what regs expression reads vs what regs statement writes. Disjoint → independent.

If memory involved: "Do two pointers point at the same thing?" This is **undecidable**.

Possible answers: Yes (definitely same), No (definitely different), **May alias** (sometimes).

Must use **false positives** (say "may alias" when they don't). Never false negatives.

Approaches:

1. **Conservative**: all memory may interfere. Safe, pessimistic.
2. **Type-based**: different types can't alias. GCC `-fstrict-aliasing`.
3. **Allocation-site**: different malloc sites → different objects. Needs data flow analysis.

For Tiger: commute function returns true/false. "return false" always works (pessimistic but correct).

Hoisting Function Calls

`f(g(x), h(y))` → t1=g(x); t2=h(y); result=f(t1,t2).

Why: Without this, nested calls trash each other's argu-

ment registers and return values (a0, v0 get overwritten). After pulling to top level, each call is isolated.

Order: left to right in Tiger. Must maintain evaluation order for side effects.

Linearizing (Block Ordering)

Goal: put basic blocks into linear sequence. Requirement: false branch of every CJUMP falls through.

BFS is bad: Execution flows go everywhere, lots of extra jumps needed. Straight-line sequences are short.

DFS is good: Execution follows one path deeply. Straight-line code sequences are longer. Fewer extra jumps.

Topological sort won't work: CFG has cycles (loops).

Branch flipping: If DFS puts "true" successor next instead of "false," flip the relop. `notRel`: negate $<$ to $\geq$, $=$ to $\neq$, etc.

Profile-guided optimization: Branches are not 50/50. Loop branches usually taken. DFS should follow likely path. GCC: `__builtin_expect(expr, val)`, C++20: `[[likely]]`/`[[unlikely]]`.

I-cache: Hot path blocks close together. Rarely-executed blocks (error handlers, epilogs) placed far away to avoid polluting instruction cache.

Instruction Selection

Maximal Munch Algorithm

1. At current node, match the **largest** IR subtree to a single asm instruction.
2. Recurse on remaining children to get operands into registers.
3. Each recursion returns the register where result lives.
4. Emit instruction using returned registers.

Two functions: **munch_stm** (statements, no register returned) and **munch_exp** (expressions, returns a register).

SML pattern matching is perfect: each pattern = one instruction template. Start simple (each node = basic instruction), then add special cases.

Key patterns:

- MEM(BINOP(+, e, CONST c)) → single `LW rd, c(rs)` (load with offset!)
- BINOP(+, e, CONST c) → `ADDI rd, rs, c` (add immediate!)
- MOVE(MEM(BINOP(+, e1, CONST c)), e2) → `SW rs, c(rd)`
- MEM(e) alone → `LW rd, 0(rs)`
- CONST(c) → `LI rd, c`

Worked: a[i]

IR: MOVE(MEM(+(a, *(i, 4))), *(+(MEM(+(z,8)), 27), 3))
Step by step (top-down match, bottom-up emit):

```
LI t100, 4    (CONST 4 – MIPS has no mul-immediate)
MUL t101, i, t100    (i*4)
ADD t102, a, t101    (a + i*4 = address)
LW t103, 8(z)    (MEM(+(z, CONST 8)) → single LW with offset!)
ADDI t104, t103, 27    (+(e, CONST 27) → ADDI!)
LI t105, 3
MUL t106, t104, t105    ((z.field3 + 27) * 3)
SW t106, 0(t102)    (store to a[i])
```

Key optimizations: MEM(+(e,CONST)) → single LW. +(e,CONST) → ADDI. Without these: 2 extra instructions.

Multiplication Optimization (Strength Reduction)

MIPS has no multiply-immediate. Must load constant first.

- Multiply by 4 (power of 2): shift left 2 (1 cycle vs 3+)
- Multiply by 3: shift left 1 + add original (2 instrs)
- Multiply by 127: shift left 7, subtract original (2 instrs)
- Many set bits: just use MUL

Cycle costs: ALU/shift = 1 cycle. Multiply = 3–4 cycles.

Output Data Types

- **OPER**: asm string with `` `s0 ``, `` `d0 `` placeholders + src/dst lists + jump targets. jump=NONE → fall through. jump=SOME([]) → end of function (JR RA). jump=SOME(list) → branch targets.
- **MOVE**: reg-to-reg move. Kept separate: if reg allocator maps src=dst, **move eliminated**. Can't do this for multiply etc.
- **LABEL**: marker.

JAL destinations: list ALL caller-saved regs (t0–t9, a0–a3, v0, ra) because callee may clobber them. Even though only v0 is "return value," call potentially overwrites all caller-saved.

Register Pressure and Ordering

Left-side address result (t102) must live across all right-side computation. Could reorder: compute right side first, then left. Safe because expressions are pure. Reduces simultaneously-live temps.

Instruction Scheduling (Not Required)

Load word followed immediately by use → pipeline stall (1–3 cycle latency). Fix: move independent instructions between load and use.

SAXPY example: Loop body has data dependency chain. Schedule for 2-wide processor by building dependency graph, picking 2 ready instructions per cycle, preferring instructions with longest outgoing chain (critical path first).

Loop unrolling: Enlarges scheduling scope (more independent instructions to interleave). Needs alias analysis to prove stores and loads from different iterations don't conflict. `restrict` keyword in C: this pointer doesn't alias others.

Limits: I-cache pressure, register pressure, diminishing returns.

Liveness Analysis

Definitions

Live: a variable's value may be read in the future before it is written again. False positives (say live when dead) are **safe** – worse register pressure but correct code. False negatives (say dead when live) are **UNSAFE** – would break code.

Dead: definitely going to be written before read, or never read again.

This is undecidable in general (some control flow paths may be impossible). We approximate.

Basic Blocks

Sequence of instructions: enter first, execute all through last. No jumps in, no jumps out of middle. Jump only at end. **Maximal** such sequence.

Def and Use Sets

For each block B:

- **def(B)**: temps/variables that B writes to
- **use(B)**: temps that B reads **before** any write to them in B

If block writes t100 then later reads t100: only DEF (doesn't need incoming value). If block reads t100 then later writes t100: USE (needs incoming value).

Liveness Equations

$\text{live-in}(B) = \text{use}(B) \cup (\text{live-out}(B) - \text{def}(B))$
$\text{live-out}(B) = \bigcup_{S \in \text{succ}(B)} \text{live-in}(S)$
Direction: Backward. **Meet**: Union over successors. **Fixed point**: Least (start with $\emptyset$, only add).

Fixed Point Iteration

1. Initialize: live-in(B) = $\emptyset$, live-out(B) = $\emptyset$ for all B
2. For each block (prefer **reverse order** for speed), compute live-in and live-out
3. If anything changed, repeat step 2
4. When nothing changes: done (fixed point reached)

Why LEAST fixed point (start from $\emptyset$): Starting full

and removing gives "greatest" – a variable is live "because it can be" not "because it has to be." Example: if you start assuming X is live in a loop, you'll conclude X is live everywhere even if never used. Least fixed point only marks live because of actual uses.

Convergence: Finite variables, sets only grow (monotonic) → must converge.

Processing order: Backwards through CFG converges faster. Forward works too, just more iterations. Same correct answer either way.

Worked: Instruction-Level Liveness

Block: `x = q + a; y = c * 3; z = 5`. live-out={x, y, u}.
Work backwards through instructions:
After `z=5`: kill z (not in set), gen nothing new. Live = {x, y, u}
After `y=c*3`: kill y, gen c → {x, c, u}
After `x=q+a`: kill x, gen q, a → {q, a, c, u} = live-in
Each instruction: remove destination (kill), add sources (gen).

Worked: Liveness with Loops (Vampire/Zombie Example)

Variables: vampires, zombies, x, y, z, buffy, answer, snack. CFG with 7+ blocks.
First pass (backward): Some blocks' successors haven't been processed yet (still ∅). Partial results.
Second pass: "Zombies" propagate everywhere through the loop back-edges. Variables used inside a loop become live throughout it. "Zombies propagate everywhere – which is what zombies do."
Third pass: Nothing changes → converged.
Key insight: Liveness propagates backwards through loop back-edges. First pass may miss variables; second pass propagates them through the cycle.
Only-add property: We never remove from sets during iteration. Once a variable enters a liveness set, it stays. This guarantees convergence.

Infeasible Paths

If `x<7` implies `x<9`, the path where `x<7` is true but `x>=9` is impossible. Liveness can't detect this → false positives. Adding analysis improves but never perfects (undecidable). "The Turing cliff is always there" – trying to be perfect makes your analyzer loop forever.

View Shift (procEntryExit)

Caller puts args in a0–a3 (extras on stack). Callee's IR expects vars in specific temps/frame slots.

View shift inserts moves: `MOVE(temp100, a0)` (x from register), `SW a1, -4(fp)` (y escapes to frame), `LW temp103, ??(sp)` (b from stack, 5th arg), etc.

Register-to-register moves might be eliminated later if allocator assigns same physical register.

procEntryExit 1/2/3: Different phases of prolog/epilog generation. 1 = view shift + save/restore callee-saved regs. 2 = sink instruction (defines callee-saved regs as live at end so allocator doesn't use them). 3 = prolog/epilog assembly (SP adjustment, label, JR RA with SOME([])).

Register Allocation

Interference Graph

Nodes = temps/variables. **Solid edge** (interference): two temps live at same time, cannot share register. **Dashed edge** (move): move-related, we'd LIKE them in same register.

Building: For each write to temp t, add interference edges between t and everything **live-out** after that instruction (except t itself). For MOVE instructions: add move edge (not interference) between src and dst.

k = number of physical registers. Goal: k-color the graph so no adjacent nodes share a color. Each color = one physical register.

Graph Coloring is NP-Complete

NP = **nondeterministic polynomial** (NOT "not polynomial"). Can verify a solution in polynomial time. P=NP? Unknown ($1M prize).

Best known exact algorithm: exponential. For ∼25 MIPS registers and typical programs, brute force would work. But we use heuristic anyway.

Safe approximation: Accept false failures (say "can't color" when actually could) → unnecessary spills but correct code. Never false success (same color on adjacent nodes → INCORRECT code).

Trivial Degree

A node has **trivial degree** if its degree (number of **interference** edges, NOT move edges) is strictly $< k$.

Key property: If the rest of the graph can be k-colored, a trivial-degree node can ALWAYS be colored. It has fewer than k neighbors, so at least one color is available.

Algorithm: Simplify → Coalesce → Freeze → Spill

1. SIMPLIFY: Remove a node of trivial degree that is NOT move-related. Push onto stack (record adjacencies). This may lower neighbors' degrees, creating new trivial-degree nodes. Repeat until no more.

Move-related nodes are NOT eligible – we want to try coalescing first.

2. COALESCE: Try to merge two move-related nodes (eliminating the register-to-register move). Only coalesce if safe (won't force spills).

Coalescing Heuristics (if EITHER says yes ⇒ safe)

Briggs: Count the combined node's neighbors of **non-trivial degree** ($\geq k$). If count $< k$, safe. Rationale: all trivial-degree neighbors can be simplified away later; only non-trivial ones matter.
George (A into B, **asymmetric**): For every neighbor T of A: either T **already interferes with B** (no new constraint), OR T has **trivial degree** ($< k$, simplifiable). Must check both directions separately – may get different answers!
If EITHER heuristic says yes in EITHER direction, coalescing is safe.

After successful coalesce, go back to SIMPLIFY (may open new opportunities).

3. FREEZE: If stuck (can't simplify or coalesce): remove a move edge (give up on that move elimination). This "unfreezes" nodes for simplification. Prefer edges where ≥1 node is trivial degree. Go back to SIMPLIFY.

4. POTENTIAL SPILL: If stuck (can't simplify, coalesce, or freeze): remove ANY node, push to stack, mark with star. When rebuilding, might not be colorable. Heuristic: low-degree = more likely to get lucky; high-degree = removes more constraints.

5. SELECT (Pop and Color): Pop nodes from stack, assign lowest available color not used by neighbors. Trivially-removed nodes = guaranteed color. Potential spills may fail → **actual spill** → insert loads/stores to stack, **restart from scratch**.

Failure does NOT prove graph is not k-colorable – just that this ordering failed. Backtracking would give correctness but exponential time.

Worked: Coalescing Check for C and D

George C→D: C's neighbors: M (interferes with D? **YES** ✓), B (interferes with D? **YES** ✓). All pass → **safe**.
George D→C: D's neighbors: M, B, K, J. K does NOT interfere with C and is NOT trivial degree → **FAILS**.
One direction passed → proceed with coalesce.
Why asymmetric: Combining A into B only adds A's neighbors to B. If all of A's neighbors are already connected to B or trivial, the combined node's degree doesn't effectively increase.

Worked: Coalescing Check for B and J

Briggs: Combined BJ node's non-trivial-degree neighbors: E (non-trivial), M (non-trivial), CD (non-trivial) = 3 ≥ k=3 → **FAILS**.
George B→J: B adj to E (interferes with J? Yes ✓), M (interferes with J? **NO**; trivial? **NO** → FAIL).
George J→B: J adj to E (✓), CD (✓), K (✓), F (interferes with B? **NO**; trivial? **NO** → FAIL).
Neither heuristic says yes → **cannot coalesce**.

Practice Final Q4 – 3-Register Machine

Nodes: x, y, z, a, b, c, p, q. k=3.
Step 1: Simplify b (trivial degree <3, no move edges) → push.
Step 2: Coalesce p,q (move-related, safe by heuristic) → merge into pq.
Step 3: Simplify pq (degree dropped <3) → push.
Step 4: Freeze x–c move edge (can't safely coalesce – combined node would have too many high-degree neighbors).
Step 5: Simplify x, y, a, z, c (all now trivial degree or non-move-related) → push each.
Pop and color: c=r1, z=r2, a=r3, y=r1, x=r2, pq=r2, b=r3.
p and q both get r2 (coalesced) → move eliminated!

Pre-colored Temps

Certain temps MUST be specific registers (temp100=a0 for calling convention). All pre-colored temps interfere with each other. **Never simplified or spilled**. Base case: only pre-colored remain.

Temp live across JAL → interferes with ALL caller-saved (t0–t9, a0–a3, v0, ra) → naturally gets s-register. Short-lived temps near calls are free to use t-registers. The algorithm handles this automatically.

S-reg saving strategy: Try 0 s-regs; if allocation fails, try 1 (s0), then 2, etc. Avoids wasteful saves. Use lowest-numbered color first (reuse registers, leave more open).

Spilling

If coloring fails: pick a variable, mark as escaping, every access becomes load/store from stack. Very short live ranges = less interference. Restart register allocation from scratch.

If fails again, spill another variable. More sophisticated: analyze where register pressure is highest, add loads/stores only there. Consider: inside a loop? Save before, load after.

Data Flow Analysis Framework

Master Table

Analysis	Dir	Meet	Init	Drives
Avail Exprs	Fwd	∩ pred	Greatest (full)	CSE
Reaching Defs	Fwd	∪ pred	Least (∅)	const/copy prop
Liveness	Bwd	∪ succ	Least (∅)	reg alloc
Very Busy	Bwd	∩ succ	Greatest (full)	code hoisting

Rule: ∩ (intersection) ⇒ greatest fixed point (start full, shrink). ∪ (union) ⇒ least fixed point (start ∅, grow).

Available Expressions

Expression x op y is **available** at a point if it has been computed on **ALL** paths reaching that point and its operands have not been modified since.

$\text{AE-in}(B) = \bigcap_{P \in \text{pred}} \text{AE-out}(P)$
$\text{AE-out}(B) = (\text{AE-in}(B) - \text{Kill}) \cup \text{Gen}$

Gen: expressions computed in B (whose operands are not redefined in B after the computation).

Kill: any expression containing a variable that B writes to. Also: write to memory kills anything using memory (unless alias analysis says safe).

Worked: Why Greatest Fixed Point for Available Exprs

Block 1: `a = x*y` → loop (doesn't modify x,y) → Block 2: `b = x*y`.
Wrong (start ∅, least fixed point): Loop back-edge merge: {x*y} ∩ {} = {}. x*y never available at Block 2. **WRONG** – can't do CSE when we should.
Right (start full, greatest fixed point): Loop back-edge merge: {everything} ∩ {x*y} = {x*y}. x*y IS available at Block 2. Converges correctly.
Why: Cycles with intersection need greatest fixed point. Starting ∅ kills everything through cycles. Starting full lets correct information survive.
General rule: ∩ analyses are like "innocent until proven guilty" – start assuming everything, remove what's contradicted. ∪ analyses are "guilty until proven innocent" – start empty, add what's confirmed.

How CSE Actually Works (Multiple Predecessors)

If x*y is available via different computations (`a=x*y` in one path, `b=x*y` in another): can't just use a or b directly (they may have changed). Solution: at each computation site, add `temp = result`. At use point, replace with `temp`. May introduce reg-to-reg moves (cheaper than multiply, may be eliminated by allocator).

Reaching Definitions

A definition "x:=…" at line L **reaches** a use of x if there exists a path from L to the use with no other definition of x.

$\text{RD-in}(B) = \bigcup_{P \in \text{pred}} \text{RD-out}(P)$
$\text{RD-out}(B) = (\text{RD-in}(B) - \text{Kill}) \cup \text{Gen}$

Gen: definitions in B (the last definition of each variable).

Kill: all OTHER definitions of the same variable anywhere in the program.

"Definition" = assignment, not declaration. Line 47 and line 12 are different definitions even if both say `x = c - d`.

Worked: Reaching Definitions Applications

1. Constant Propagation: Only reaching def is `x=0`, have `a=x+q` → `a=0+q=q`. Then constant folding simplifies.
2. Copy/Move Propagation: Only reaching def is `x=y` → replace all uses of x with y. Then `x=y` becomes dead code → delete.
3. Dead Code Elimination: Definition reaches NO uses and has no side effects → delete it.
4. Uninitialized Variable Detection: No definitions reach a use → variable may be uninitialized (compiler warning).
5. Live Range Splitting: See below.

Practice Final Q5 – Reaching Definitions

CFG: Block A (instrs 1–3) → B (4–7) → C (8–10) or D (11–14) → back to B or E (15=return).
Defs reaching use of a in instr 4: **1, 8** (def at 1 from Block A, def at 8 from Block C; note: solution key shows 1,8)
Defs reaching use of a in instr 8: **1, 8, 13** (from A via B, from C looping back, from D's def at 13 via B then C)
Defs reaching use of b in instr 6: **2, 11** (def at 2 from A, def at 11 from D looping back)

Live Range Splitting

Label all defs (D0, D1…) and uses (A, B…) of a variable. For each use, find reaching defs. Build graph connecting uses to their reaching defs. Each **connected component** = separate live range → rename to different temps → lower interference.

Example: Two for-loops both using `i`. Defs D0,D1 reach uses A–D (loop 1). Defs D2,D3 reach uses E–H (loop 2). Two disjoint components → split into `i` and `i'`.

Very Busy Expressions

Expression is **very busy** at a point if it will **definitely be computed** on ALL forward paths before any operand is modified.

$\text{VBE-in}(B) = (\text{VBE-out}(B) - \text{Kill}) \cup \text{Gen}$
$\text{VBE-out}(B) = \bigcap_{S \in \text{succ}} \text{VBE-in}(S)$
Backward, intersection, greatest fixed point.

Drives: **Code hoisting** (precompute expression earlier – reduces code size, helps I-cache). Also LICM safety check (expression guaranteed to execute → safe to hoist).

Exception behavior warning: Moving `x/y` earlier could change when divide-by-zero occurs. Changes exception behavior = confusing to programmer.

Practice Final Q5 – Matching

CSE	Available Expressions
Forward flow / union	Reaching Definitions
Def/Use web formation	Reaching Definitions
Backwards flow / intersection	Very Busy Expressions

Practice Final Q5 – Very Busy Expressions Table

Block	a+1	m-1	a+b	b*47	x+y	b+1	arr[b]	
After 3	Y	N	Y	Y	N	N	N	
After 7	Y	Y	N	N	N	N	N	After
After 10	N	N	N	N	N	N	N	
After 14	Y	Y	N	N	N	N	N	
After 15	N	N	N	N	N	N	N	

block 3: a+1 is Y (both C and D compute a+1); a+b is Y (instr 4 computes on all paths); b*47 is Y (instr 6). After block 7: a+1 Y (both branches); m-1 Y (block C). After 15: all N (nothing after return).

Compiler Optimizations

Worked: Quicksort Partition Optimization

Naive code computes `4*i` five times, `4*j` multiple times.
1. Strength Reduction (instruction selection): `4*i` → `i<<2`. Mul→shift.
2. CSE (Available Expressions): Compute `4*i` once as t2 in B2. Is `4*i` available at other uses? i not changed between B2→B5 → YES, reuse t2. Eliminates 4 multiplies. Same for `4*j` (reuse t4) and `4*high` (reuse t1).
3. Move/Copy Elimination (Reaching Defs): After CSE, t6=t2 (was 4*i). Only reaching def → replace all uses of t6 with t2. Delete t6=t2 (dead code).
4. Reuse Loads: a[t2] loaded in B2 (into t3). No stores between B2 and B5 → reuse t3. B5 swap: 9 instructions → 3 (just stores + branch).
5. Alias Gotcha: Can't reuse a[t1] (a[high]) in B6 because swap in B5 writes to a[t2] and a[t4] which *might* equal a[t1]. Compiler can't prove they don't alias.
6. Induction Variable Elimination: i only changes as i=i+1 and only used as 4*i → introduce t2 that increments by 4 each iteration. Replace 4*i with t2 everywhere. Similarly j → t4−=4.
7. Comparison Replacement: Replace `i >= j` with `t2 >= t4`. WARNING: `4*i >= 4*j` ≠ `i >= j` in fixed-width arithmetic due to overflow! But 4*i is an array index – overflow would segfault first → safe here, but "something to be wary of in general."

Loop Invariant Code Motion (LICM)

All defs reaching operands of expression come from outside loop AND expression has no side effects → can hoist before loop.

Worked: LICM Danger – Zero-Trip Loops

`for(i) for(j in 0..m) total += a[i][j]`
`a[i]` is loop-invariant w.r.t. inner loop → want to hoist `p = a[i]`.
Danger: If m=0 and a=null, original code never executes `a[i]` (inner loop body never runs). Hoisting causes **segfault** in working code!
Fix: Convert while-loop to `if(m>0) { do { ... } while(...) }`. Now loop body executes at least once. `a[i]` is a **very busy expression** in the do-while body (guaranteed to execute) → hoisting is safe.
This is where Very Busy Expressions analysis is useful!

Strength reduction: `4*i` → `i`≪2. Mul by 3 → shift+add. Mul by 127 → shift−sub.

Domination & Loop Identification

Domination

A **dominates** B: ALL paths from start to B pass through A. Every node dominates itself.

$\text{Dom}(B) = \{B\} \cup \bigcap_{P \in \text{pred}} \text{Dom}(P)$

Entry node: Dom(entry) = {entry}. All others: init to universal set. Intersection ⇒ greatest fixed point.

idom(B): closest strict dominator (≠ B itself). The immediate dominator. Dom tree: path from root to B = all of B's dominators.

Back Edges and Natural Loops

Back edge: edge $E \to S$ where S **dominates** E. Each back edge defines a **natural loop**. The destination S is the loop **header** (entry point).

NOT a loop if target doesn't dominate source: Two nodes with edges both ways but neither dominating the other = NOT a proper loop (no clear entry point). This pattern comes from arbitrary goto's, not normal for/while.

The header dominates the entire loop body – can't enter the loop without going through the header.

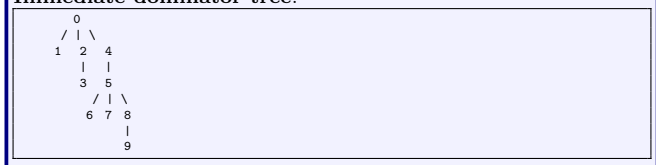
Practice Final Q6 – Domination (10 nodes)

CFG: 0→{1,2}, 1→4, 2→3, 3→{3,4}, 4→{1,5}, 5→{6,7}, 6→8, 7→8, 8→{4,9}.

Dominator sets:
Dom(0) = {0}
Dom(1) = {0,1}
Dom(2) = {0,2}
Dom(3) = {0,2,3}
Dom(4) = {0,4}
Dom(5) = {0,4,5}
Dom(6) = {0,4,5,6}
Dom(7) = {0,4,5,7}
Dom(8) = {0,4,5,8}
Dom(9) = {0,4,5,8,9}

Key: Dom(4)={0,4} because two paths reach 4: 0→1→4 and 0→2→3→4. Since 1 is not on the second path and 2,3 not on the first, only 0 (and 4 itself) dominate 4.
Back edges (loops):
- 3→3: self-loop (3 dominates 3 ✓)
- 8→4: loop body = {4,5,6,7,8} (4 dominates 8 ✓)

4→1 is NOT a loop: 1 does NOT dominate 4 (path 0→2→3→4 bypasses 1). Edge looks like a loop but fails domination test.
Immediate dominator tree:



idom(1)=0, idom(2)=0, idom(4)=0, idom(3)=2, idom(5)=4, idom(6)=5, idom(7)=5, idom(8)=5, idom(9)=8.

Garbage Collection

Garbage: unreachable memory (safe approximation of "can't affect future computation," which is undecidable).

Reference Counting

Each object has a count of incoming pointers. When count hits 0, free (recursively decrement children).

Worked: Code for p

```
// Step 1: Increment q's refcount
if (q != null) {
    temp = q->refcount;
    temp++;
    q->refcount = temp;
}
// Step 2: Decrement p's refcount
if (p != null) {
    temp = p->refcount;
    temp--;
    if (temp == 0)
        recursive_free(p); // free p and its children
    else
        p->refcount = temp;
}
// Step 3: Do the actual assignment
p = q;
```

A register-to-register move becomes: ∼3 branches, 2 loads, 2 stores, 2 arithmetic ops, 1 move. "And a partridge in a pear tree."
In multithreaded code: need atomic ops on refcounts (∼50 cycles each). Overhead is huge for pointer-heavy code.
MUST increment q first, then decrement p. If p and q point to the same object and you decrement first, refcount hits 0 and the object is freed before you can increment!

Pros: Simple, incremental (no stop-the-world).

Cons: Very slow (∼20–30% overhead). **Cannot collect cycles** (A→B→A stays refcount≥1 forever). 1 word per object overhead. Python uses it.

Mark and Sweep

Root set: pointers in registers, stack, globals.

Mark phase: DFS from root set, mark every reachable object (mark bit = visited set, handles cycles).

Sweep phase: Walk heap **linearly** using size headers. Unmarked → free list. Marked → clear mark bit.

Cost: O(live + dead) – must touch ALL objects during sweep. Must **stop the world**.

Space: 1 word per object (size header / mark bit). Causes **fragmentation** (freed objects leave holes).

Making heap bigger does NOT help much (numerator and denominator both grow in amortized analysis).

Pointer Identification

How does GC know which fields are pointers?

1. **Compiler type info / bitmasks**: Object header has bitmask (1 bit per field: pointer or not). Stack frames have similar headers. Register file: lookup table keyed by PC (return address) → 32-bit bitmask of which registers hold pointers.
2. **Conservative collection**: Anything that looks like a valid heap address = maybe pointer. **Downside**: can't move objects (non-pointer integer treated as pointer would become invalid if you move the "target").
3. **Tagging**: Steal 1 bit per word (low bit: 0=pointer since pointers are word-aligned, 1=integer). **Downside**: reduces integer range by 1 bit.

Stop and Copy

Worked: Stop-and-Copy Algorithm

Heap split into **active** and **inactive** halves. Allocate in active half via **bump pointer** (just increment – as fast as stack allocation!).
When active half is full:
1. DFS from root set
2. Copy each live object to inactive half
3. Leave **forwarding pointer** in old location
4. When encountering an already-copied object (has forwarding ptr), just update the reference to point to the copy
5. Dead objects are **never touched** – they just get overwritten when halves swap

Example: Roots point to D. Copy D to inactive, leave fwd ptr. D refs B → copy B, leave fwd ptr. B refs A → copy A. G refs B → B already copied (fwd ptr) → update link. C, E, F are dead – never touched!
Swap halves. Old half = meaningless bits.

Pros: O(live) only – dead objects cost nothing! No fragmentation (automatic compaction). Super fast allocation (bump pointer). Good locality (referenced objects copied adjacent).

Cons: **Wastes half the heap.** Copying cost (but most objects small). All objects move to new addresses (must update all pointers).

Generational GC

Key insight (infant mortality): Most objects die young. Long-lived objects tend to keep living.

Multiple generations:

- **Gen 0 (nursery)**: ∼256KB (fits L2 cache!). Stop-and-copy. Collected very frequently. ∼95% is garbage → very fast.
- **Gen 1**: ∼4MB. Mark-and-sweep. Fills slowly.
- **Gen 2**: ∼6GB. Collected rarely.

Objects start in Gen 0. After surviving N collections (2-bit counter in header, e.g., survive 3 times), promote to Gen 1. Similar promotion to Gen 2.

Size Gen 0 to fit L2 cache: If Gen 0 = 256KB and L2 = 512KB, all copying is cache hits.

Old→young pointers problem: Must find old objects pointing at young objects (they're roots for young-gen collection). Scanning all old objects defeats the purpose.

- **Purely functional** (Haskell): Objects only point at older objects. Problem can't occur.
- **Imperative**: `mprotect()` marks old pages read-only. Writes trigger SIGSEGV, handler records dirty page, marks writable. Dirty pages scanned during young-gen collection.

Promoting referenced objects: When moving D to older gen, also move B and A (objects D references) – likely long-lived too.

Java permanent generation: Class objects, method objects that never die → special region never GC'd.

Practice Final Q7 – Garbage Collection

3 advantages of GC: (1) Fast allocation (S&C bump pointer), (2) Improved locality (S&C compacts), (3) No double frees, (4) No use-after-free/dangling ptrs, (5) No memory leaks, (6) Reduced dev/debug time. (Must specify which algorithm if advantage is algorithm-specific.)
Tradeoffs:
- RefCount: Very high per-op overhead (∼20–30%), 1 word/obj, can't collect cycles
- M&S: O(live+dead), 1 word/obj, fragmentation, stop-the-world
- S&C: O(live only), **wastes half heap**, compacts, fast alloc

Pointer ID methods: (1) Compiler type info/bitmasks, (2) Conservative (can't move objects), (3) Tagging (steal 1 bit/word, lose integer range).

Compiling OO Languages

Polymorphism Types

1. **Ad-hoc / overloading**: Same name, different implementations. C++ name mangling (`add_int_int` vs `add_float_float`).
2. **Parametric**: C++ templates (specialize per type, separate code) vs Java erasure (compile as Object, type param not available at runtime, can't do `new T()`).
3. **Subtype**: vtables and dynamic dispatch.

VTables & Dynamic Dispatch

Putting function pointers in every object: space = methods × objects. Too expensive.

Instead: each object has **one pointer** to shared class vtable. All instances share one vtable. Space = objects + methods.

Worked: Dynamic Dispatch Mechanism

Given `p->speak()`:

```
v = load_word(p, 0)      // get vtable pointer
f = load_word(v, offset) // get method pointer from vtable
a0 = p                   // pass 'this' pointer
jalr f                   // indirect call
```

Cost: 2 extra load instructions vs static dispatch.
Why C++ requires `virtual`: In 1980s, 12 MHz computers. Two extra loads per call was expensive. Opt-in with `virtual`. **Danger**: If you forget `virtual` in parent but override in child → silent bug (static dispatch used, child's method never called).
Why Java makes everything dynamic: By 1990s, HW faster. 99.9% of time, not having dynamic dispatch with inheritance is a bug. Java removes the footgun.

Single Inheritance & Sub-object Rule

Subclass must contain superclass at **same offsets**. Cat extends Animal: [vtable | name | age | laziness]. First 3 fields = valid Animal sub-object.

Animal code accessing `p->name` as `load(p, 4)` works identically whether p is Animal or Cat. Cat's vtable has same layout as Animal's for inherited methods, with overrides pointing to Cat's code.

Multiple Inheritance

Sub-objects stacked: [cat_vtable | name | age | laziness | robot_vtable | robot_name | model]. **Two vtable pointers**.

Primary parent (Cat) at top: casting to Cat/Animal is free (pointer stays same).

Non-primary parent (Robot): casting requires **pointer adjustment**: `f(p + offset)` instead of `f(p)`.

Thunks: If CyborgCat overrides Robot's method M2, Robot's vtable entry for M2 points to a **fix-up thunk**: `this -= 24; jump to actual M2 code`. The thunk adjusts `this` from robot sub-object to full CyborgCat object. Non-overridden methods point directly to original code (no fix-up needed).

Diamond Problem / Virtual Inheritance

Two parents sharing grandparent → two copies of grandparent fields (bug-prone).

C++ **virtual inheritance**: shared base placed at END of object. Only one copy of shared fields. Accessed via **offset stored in vtable** → extra indirection per access. Position varies depending on actual class of object, hence vtable lookup.

Cost: Every access to virtually-inherited fields requires extra vtable lookup.

Interface Dispatch (Java)

Interfaces have no fields. First Java: `invoke_interface` did linear search through vtable (O(n) per call, terrible).

Better: **graph coloring on vtable slots**. Methods from same interface get same slot positions across all implementing classes. Easy because can always add more vtable slots (more colors available). Dynamic class loading makes this trickier.

Hot/Cold Field Splitting

Objects with many fields: put hot (frequently accessed) fields in object, cold fields behind a pointer. Much better cache locality. Accessing cold fields: extra pointer dereference + likely cache miss. Determine hotness via profiling / JIT statistics.

JIT Compilation (Java)

javac → bytecode (stack-based ISA for JVM) → options:

1. **Interpret**: slow
2. **Compile everything AOT**: slow startup (huge stan-

dard library)

3. **JIT**: Interpret first, compile after seeing code multiple times. Only compiles what's actually needed. Gathers runtime statistics while interpreting, optimizes based on actual behavior. Can recompile/deoptimize at any time.

Miscellaneous Exam Topics

procEntryExit 1/2/3

- **1**: View shift (move args from a0–a3/stack into temps/frame slots IR expects) + save/restore callee-saved regs.
- **2**: Sink instruction – defines callee-saved regs + return-value regs as "live at end" so allocator doesn't use them for other values.
- **3**: Assembly prolog/epilog: function label, SP adjustment, JR RA with SOME([]).

Undecidability Pattern

Alias analysis, precise liveness, optimal register allocation: all undecidable/NP-hard. Approximate safely:

- Liveness false positives: extra register pressure
- Alias false positives: miss optimizations
- Reg alloc failure: spill to memory

"The Turing cliff is always there" – trying to be perfect makes your analyzer loop forever.

CPS

Confirmed NOT on exam. (But: no function returns; every function takes continuation K. Tail call = pass K directly. Exceptions = two continuations. Used as IR in SML/NJ.)

Optimization	Driven by
CSE	Available Expressions
Const/Copy propagation	Reaching Definitions
Dead code elimination	Reaching Defs + Liveness
Live range splitting	Reaching Definitions
Register allocation	Liveness
Code hoisting	Very Busy Expressions
LICM safety	Very Busy + Domination
Induction var elimination	Loop detection (back edges)